CS 423
Operating System Design: Persistence: IO Scheduling and RAID
Nov 3

Ram Alagappan

AGENDA / LEARNING OUTCOMES

IO Scheduling (brief overview)

RAID -

- motivation
- different levels
- performance characteristics

RECAP

IO devices

DMA vs. Programmed IO
Polling vs. interrupt

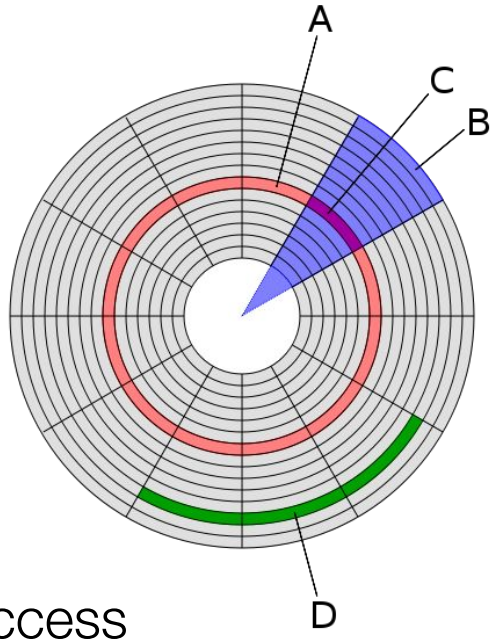
Disks: specific geometry with platters, spindle, tracks, sector, head, etc

DAT = seek time + rotation delay + transfer time

Sequential bandwidth is much higher than random bandwidth

DISK SCHEDULING

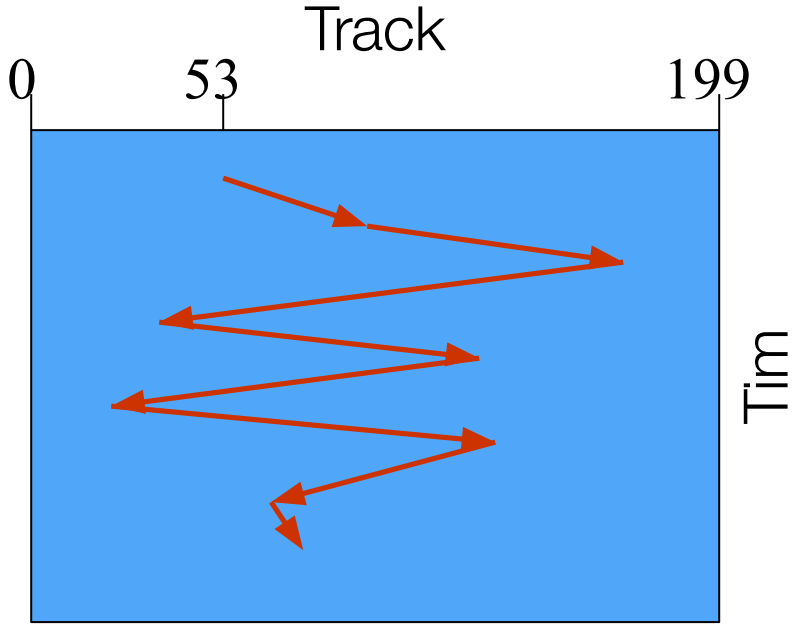
- Which disk request is serviced first?
  - FCFS
  - Shortest seek time first
  - SCAN (Elevator)
  - C-SCAN (Circular SCAN)



Disk Scheduling Decision — Given a series of access requests, on which track should the disk arm be placed next to maximize fairness, throughput, etc?

FCFS

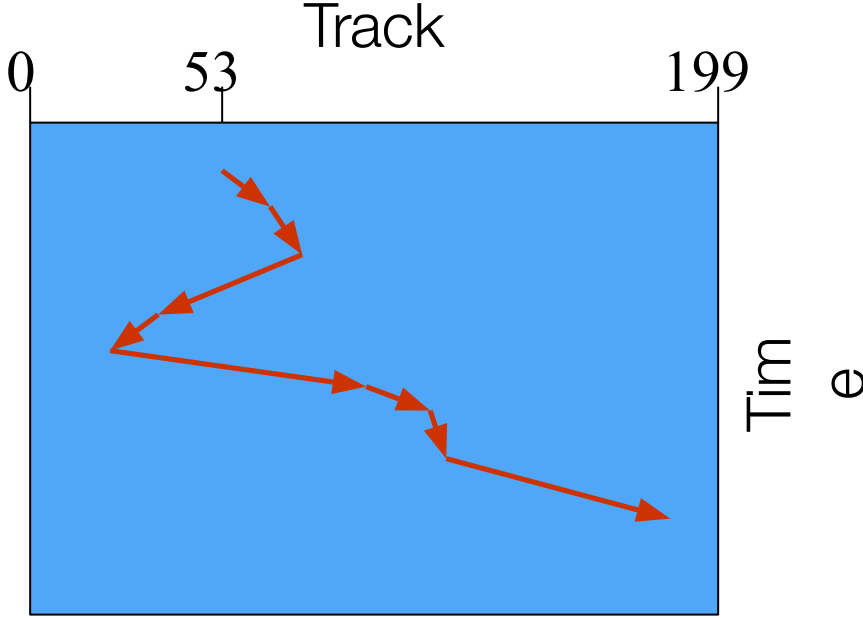
- Method
  - First come first serve
- Pros?
  - Fairness among requests
  - In the order applications expect
- Cons?
  - Arrival may be on random spots on the disk (long seeks)
  - When is it particularly bad?



98, 183, 37, 122, 14, 124, 65, 67

SSTF (SHORTEST SEEK TIME FIRST)

- Method
  - Pick the one closest on disk (greedy approach)
- Pros?
  - Tries to minimize seek time
- Cons?
  - ???

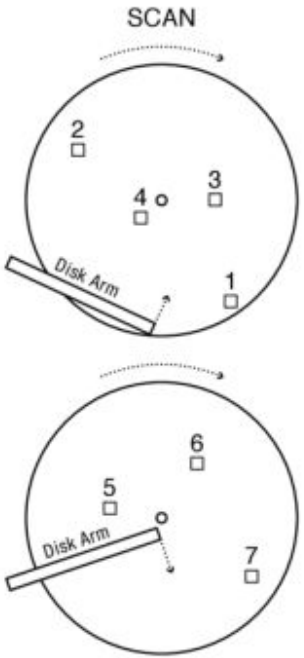


98, 183, 37, 122, 14, 124, 65, 67
(65, 67, 37, 14, 98, 122, 124, 183)

ELEVATOR (SCAN)

- Move outer to inner – service all requests along the way
- Move inner to outer – service all along the way

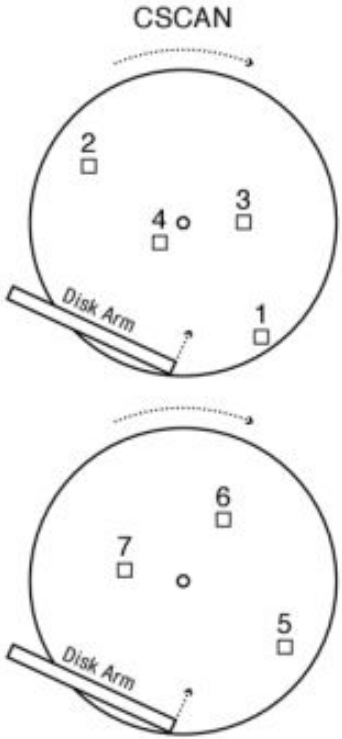
- Adv compared to SSTF:
  - Bounded time for each request



CIRCULAR SCAN (C-SCAN)

Like SCAN
But, wrap around (i.e., only one direction)

- Adv over SCAN
  - By seeking to opposite side, moves head to where pending requests are likely to be denser
  - More fair
- Cons
  - Do nothing on the return (i.e., higher overhead)

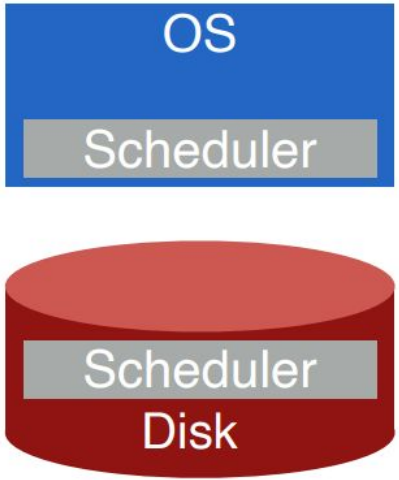


Who does the Scheduling?

The OS?

The disk itself?

Both?



Linux I/O Schedulers

- What disk (I/O) schedulers are available in Linux?

```
$ cat /sys/block/sda/queue/scheduler
mq-deadline bfq [none]
```

```
$ echo SCHEDNAME > /sys/block/DEV/queue/scheduler
```

SATA/SAS disks and some SSDs: often `mq-deadline (multi-queue)`

sometimes `bfq (budget fair queuing)`

Linux NOOP Scheduler

- Insert all incoming I/O requests into a simple FIFO
- Merges duplicate requests
- When would this scheduler be useful?

Linux NOOP Scheduler

- Insert all incoming I/O requests into a simple FIFO
- Merges duplicate requests (results can be cached)
- When would this be useful?
  - Solid State Drives! Avoids scheduling overhead
  - Scheduling is handled at a lower layer of the I/O stack (e.g., Disk firmware, RAID Controller, Network-Attached)
  - Host doesn't actually know details of sector positions

Linux Deadline Scheduler

- Imposes a deadline on all I/O operations to prevent starvation of requests
- Maintains 4 queues:
  - 2 Sorted Queues (R, W), order by Sector
  - 2 Deadline Queues (R, W), order by Exp Time
- Scheduling Decision:
  - Check if 1st request in deadline queue has expired.
  - Otherwise, serve request(s) from Sorted Queue.
  - Prioritizes reads (DL=500ms) over writes (DL=5s) .Why?

WHAT HAPPENS?

Assume 2 processes each calling read() with C-SCAN

```
void reader(int fd) {
char buf[1024]; int rv;
while((rv = read(fd, buf)) != 0)  {
assert(rv);
// takes short time, e.g., 1ms
process(buf, rv);
}}
```

P1: read 100, 101

P2: read 900, 901

After 1 ms

P1: read 102, 103

P2: read 902, 903

WORK CONSERVATION

Work conserving schedulers always try to do work if there's work to be done

Sometimes, it's **better to wait** instead if system anticipates another request will arrive

Possible improvements from I/O merging

LINUX ANTICIPATORY SCHEDULER

- Deceptive Idleness: A process appears to be finished reading from disk, but is actually processing data. Another (nearby) request is coming soon!

- Anticipatory Scheduling: Idle for a few milliseconds after a read operation in *anticipation* of another close-by read request.

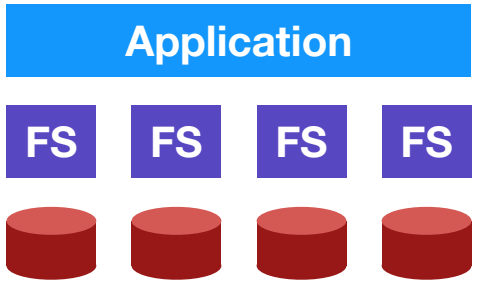
ONLY ONE DISK?

Sometimes we want many disks — why?

- capacity
- reliability
- performance

Challenge: most file systems work on only one disk

SOLUTION 1: JBOD



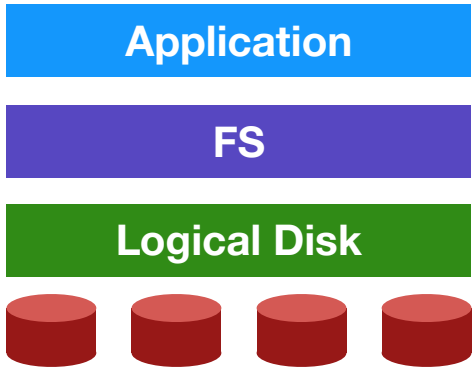
Application is smart, stores different files on different file systems.

JBOD: **J**ust a **B**unch **O**f **D**isks

SOLUTION 2: RAID

Logical disk gives capacity, performance, reliability

Build logical disk from many physical disks.



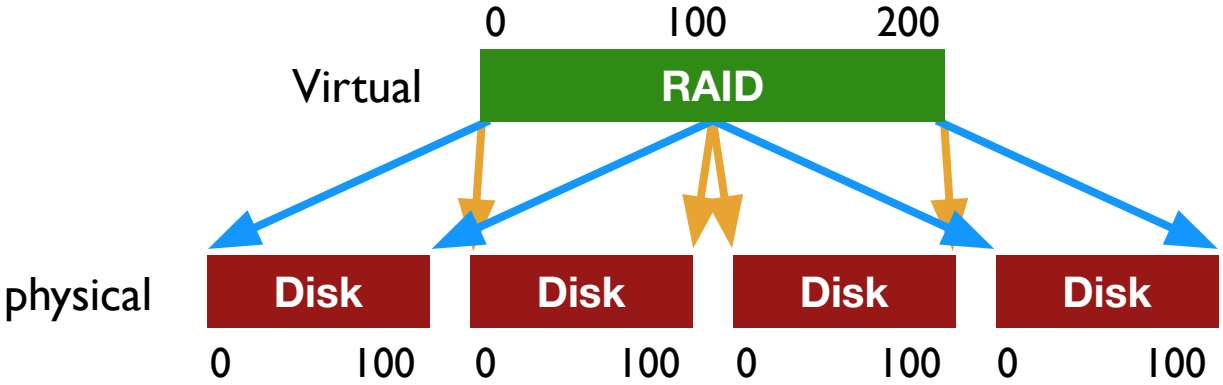
RAID usually comes a hardware box - use regular interface to connect to host
But internally, complex! has processors, memory, etc.

RAID: **R**edundant **A**rray of **I**nexpensive **D**isks

Transparency: no changes to the FS, host, Apps → ease of deployment (really important for adoption!)

D. Patterson, G. Gibson, R. Katz. SIGMOD 1988.

GENERAL STRATEGY: MAPPING, REDUNDANCY



FAULT MODEL ASSUMPTION

A disk can either work or not work

“Fail stop” model

Can detect when the disk is not working

No silent errors like LSE or block corruptions! (these are much harder to deal with)

lots of research on this topic

WORKLOADS

- Reads
  - Latency of one operation
  - Throughput of steady-state I/O
    - Sequential
    - Random
- Writes
  - Latency of one operation
  - Throughput of steady-state I/O
    - Sequential
    - Random

METRICS

Capacity: how much space can apps use?

Reliability: how many disks can we safely lose? (assume fail stop)

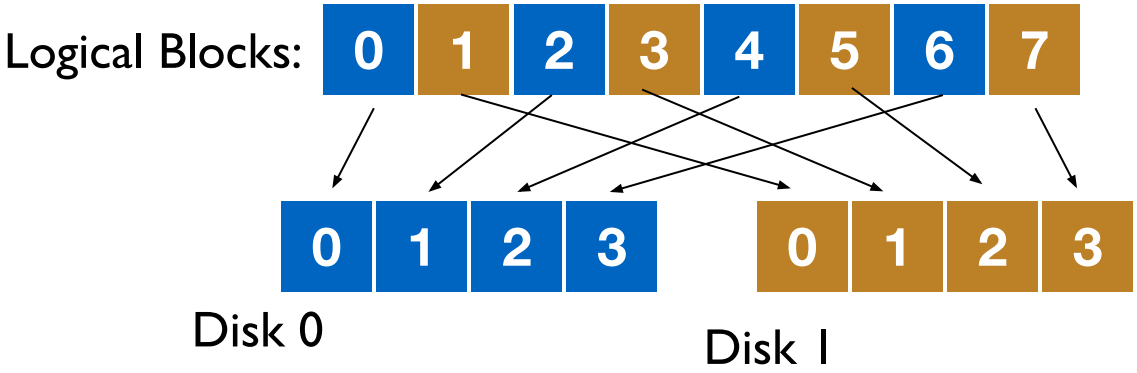
Performance: how long does each workload take? (latency, throughput)

Normalize each to characteristics of one disk

Different **RAID levels** make different trade-offs (Level 0 to Level 5)

RAID-0: STRIPING

Optimize for capacity. No redundancy



RAID 0: STRIPES AND CHUNK SIZE

Chunk size = 1

stripe:

Disk 0	Disk 1	Disk 2	Disk 4
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

assume chunk size of 1 for this lecture

Chunk size = 2

Disk 0	Disk 1	Disk 2	Disk 4
0 1	2 3	4 5	6 7
8 9	10 11	12 13	14 15

RAID-0: ANALYSIS

What is the capacity?

How many disks can we safely lose?

Latency (random)

Throughput (sequential, random)?

Discuss for two mins…

N := number of disks
C := capacity of 1 disk
S := sequential throughput of 1 disk
R := random throughput of 1 disk (usually R << S)
D := latency of one small I/O operation (read or write)

RAID 0

capacity	
how many failures can be tolerated	
latency	
sequential throughput	
random throughput	

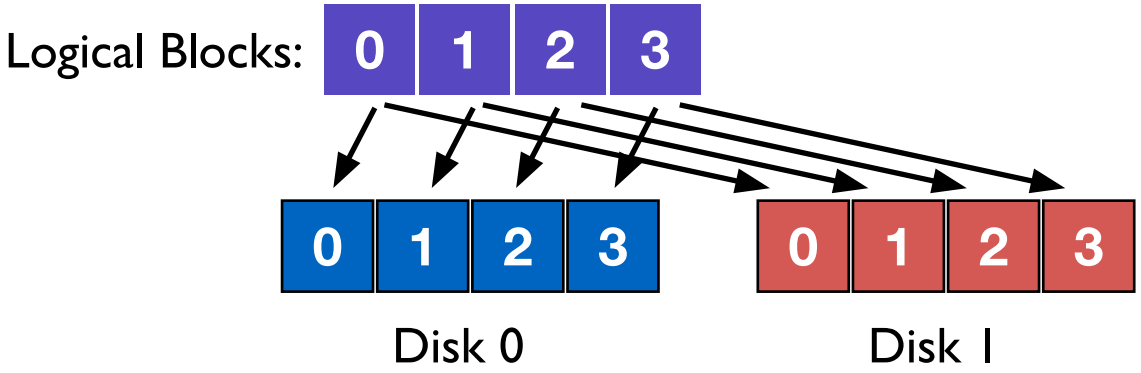
Adding more disks to a RAID 0 array improves capacity. T or F?

Adding more disks to a RAID 0 array improves throughput. T or F?

Adding more disks to a RAID 0 array improves latency. T or F?

RAID 0 latency is almost the same as standalone disk latency. T or F?

RAID-1: MIRRORING



Keep two copies of all data.

RAID-1 LAYOUT: MIRRORING

2 disks

Disk 0	Disk 1
0	0
1	1
2	2
3	3

4 disks
Sometimes referred to as RAID 10
stripe of mirrors

Disk 0	Disk 1	Disk 2	Disk 4
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

RAID-1: ANALYSIS

What is the capacity?

How many disks can fail?

Latency (read, write)?

In practice, latency of random write would be slightly higher than the latency of random write in a single disk. Why?

N := number of disks
C := capacity of 1 disk
S := sequential throughput of 1 disk
R := random throughput of 1 disk
D := latency of one small I/O operation

Disk 0	Disk 1
0	0
1	1
2	2
3	3

RAID-1: THROUGHPUT

What is steady-state throughput for

- random reads?

- random writes?

- sequential writes?

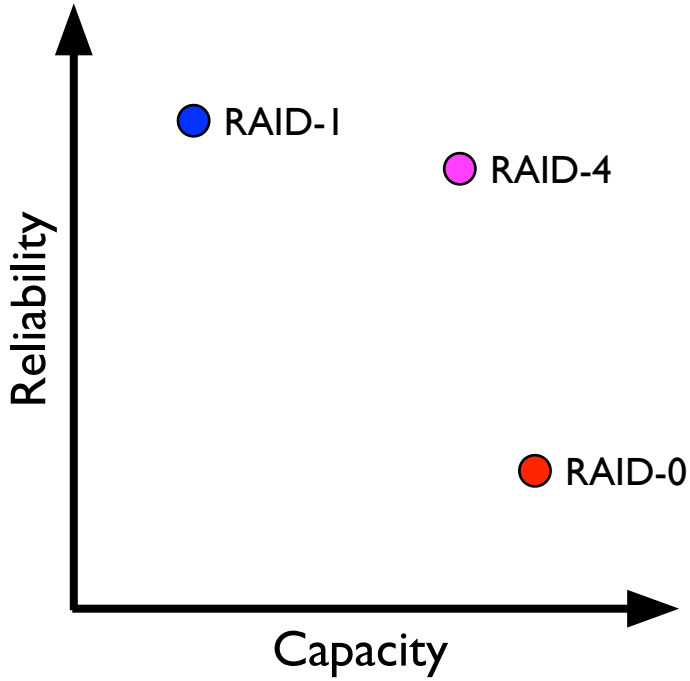
- sequential reads?

CRASH CONSISTENT UPDATES

In RAID 1 mirroring, the RAID system must exercise care when updating data

A power failure or crash can happen, leaving the copies in an inconsistent state
Example: copy on one disk update but not the other

A general systems problem
One solution: WAL (write ahead log)
Usually, the NVRAM of RAID system to avoid logging to disk overhead


RAID-1
RAID-4
RAID-0
Reliability
Capacity

RAID-4 STRATEGY

Use **parity** disk

If an equation has N variables, and N-1 are known,
you can solve for the unknown.

Treat sectors across disks in a stripe as an equation.

Data on bad disk is like an unknown in the equation.

RAID 4: EXAMPLE

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

What functions can we use to compute parity?

C0	C1	C2	C3	P
0	0	1	1	XOR(0,0,1,1) = 0
0	1	0	0	XOR(0,1,0,0) = 1

RAID-4: ANALYSIS

What is the capacity?

How many disks can fail?

Latency (read, write)?

N := number of disks
C := capacity of 1 disk
S := sequential throughput of 1 disk
R := random throughput of 1 disk
D := latency of one small I/O operation

RAID-4: THROUGHPUT

What is steady-state throughput for

- sequential reads?
- sequential writes?
- random reads?
- random writes? (next page!)

RAID-4: ADDITIVE VS SUBTRACTIVE

C0	C1	C2	C3	P0
0	0	1	1	XOR(0,0,1,1)

Additive Parity:
read all old blocks, recalculate parity block value

Subtractive Parity
Idea:
if old and new same, then xor
parity would not change, so keep it as is

if new data bit and old data bit are different, flip the parity

$$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}$$

RAID-4: RANDOM WRITE PERFORMANCE

C0	C1	C2	C3	P0
0	0	1	1	XOR(0,0,1,1)

Subtractive Parity

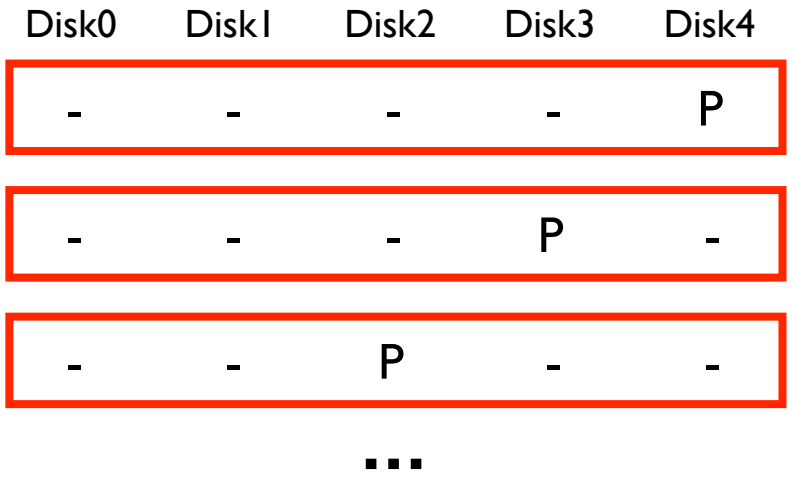
$$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}$$

Assume subtractive parity

Think which is the bottleneck here with many random IOs? By how much?

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
*4	5	6	7	+P1
8	9	10	11	P2
12	*13	14	15	+P3

RAID-5



Rotate parity across different disks

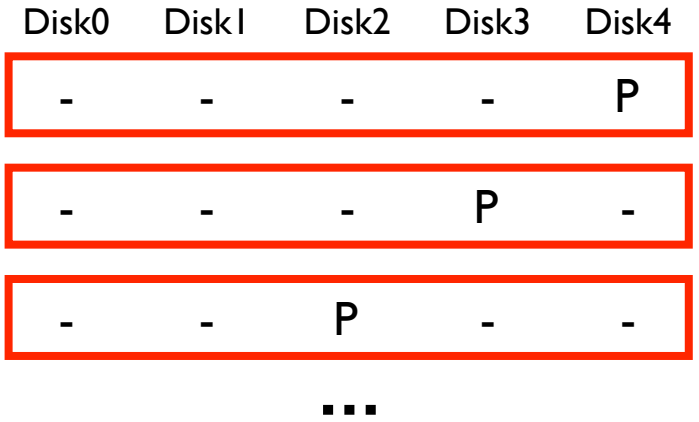
RAID-5: ANALYSIS

What is capacity?

How many disks can fail?

Latency (read, write)?

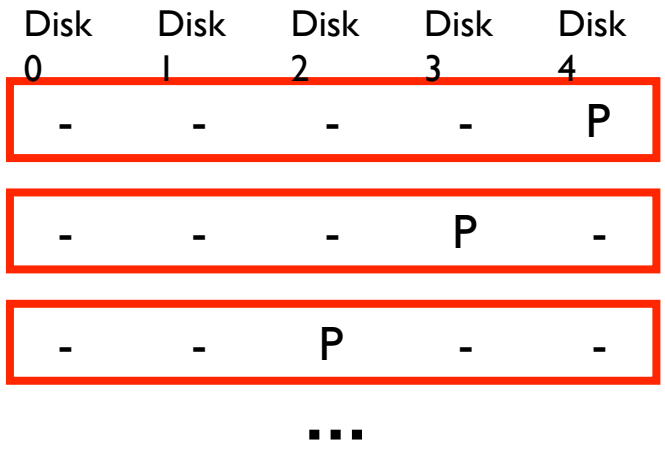
N := number of disks
C := capacity of 1 disk
S := sequential throughput of 1 disk
R := random throughput of 1 disk
D := latency of one small I/O operation



RAID-5: THROUGHPUT

What is steady-state throughput for RAID-5?

- sequential reads?
- sequential writes?
- random reads?
- random writes? (next page!)



RAID-5 RANDOM WRITES

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

RAID LEVEL COMPARISONS

	Reliability	Capacity	Read latency	Write Latency	Seq Read	Seq Write	Rand Read	Rand Write
RAID-0	0	C*N	D	D	N * S	N * S	N * R	N * R
RAID-1	1	C*N/2	D	D	N/2 * S	N/2 * S	N * R	N/2 * R
RAID-4	1	(N-1) * C	D	2D	(N-1)*S	(N-1)*S	(N-1)*R	R/2
RAID-5	1	(N-1) * C	D	2D	(N-1)*S	(N-1)*S	N * R	N/4 * R

NEXT STEPS

We are mostly done talking about device layer

We will go up the stack and talk about file systems

The prominent OS component for handling persistent storage

Lots of ways to design a FS! We will talk about a few…